

Student: Ela Debreteni

Modelling microservices with DDD

Project theme based on **Example project idea 1** (a system for auctioning with microservices operating with several bidders)

Project description: The user sends requests containing the name of the product to be sold in auction. The sales manager generates a random initial price for it and notifies the bidders. Each bidder will choose an amount to bid, higher than the initial price. Each bidder has a random initial budget. If the amount is more than their budget, then the bidder will skip the auction for that product and will reset its budget so that the simulation can continue indefinitely. If the amount is within budget, the bidder will notify the auction manager about the bid and deduct the amount from its budget. The auction will be open for 10 seconds for each product starting from the first bid on that product. Then, the auction manager will declare the winner of the auction to be the bidder with the highest amount bided. In case two bidders bid the same amount, the first one received by the auction manager will be considered.

Domain: The domain is the auctioning process

Subdomains: Sales Management, Bidding, Auction Management

Bounded Context:

Sales Context: Manages products for sale, including their initial prices.

Bids Context: Manages the bided amount and the bidder's budget.

Auction Context: Manages the auction and declares the winner.

Aggregates:

Product Aggregate: Represents a product, including its description and initial price.

Bid Aggregate: Represents a bid, including the bidder id, the product id and the amount.

Auction Aggregate: Represents an auction, including the product id, winning bidder id and winning amount.

Services: One microservice per Bounded Context

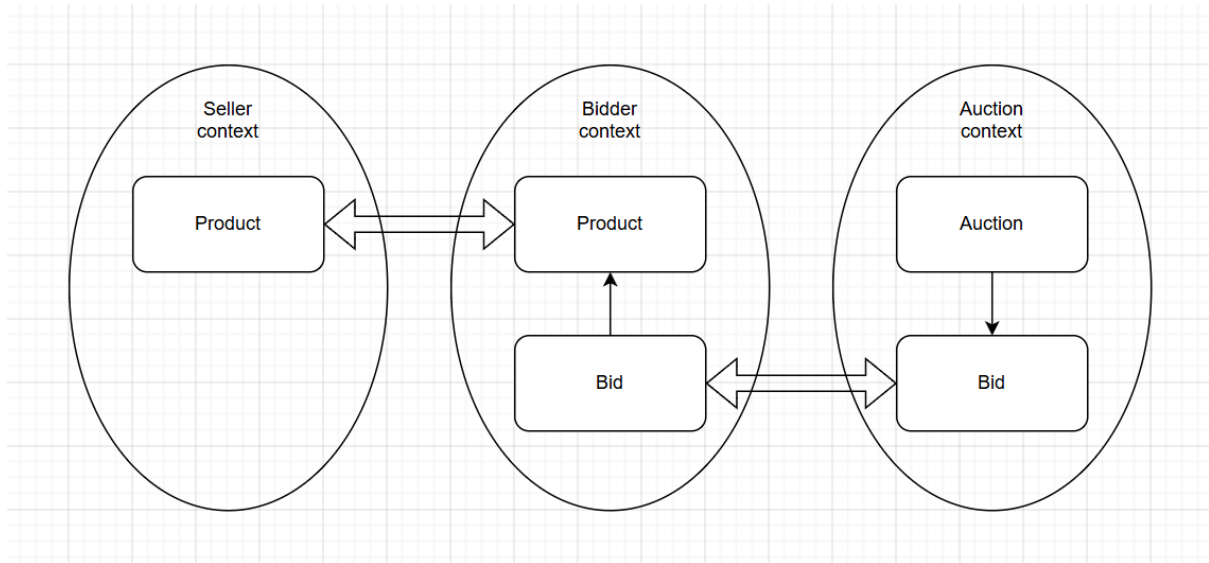


Figure 1 - Contexts and Aggregates

Flow: User-request -> Seller-service -> Product-for-sale-event -> Bidder-service -> Bid-event -> Auction-manager-service

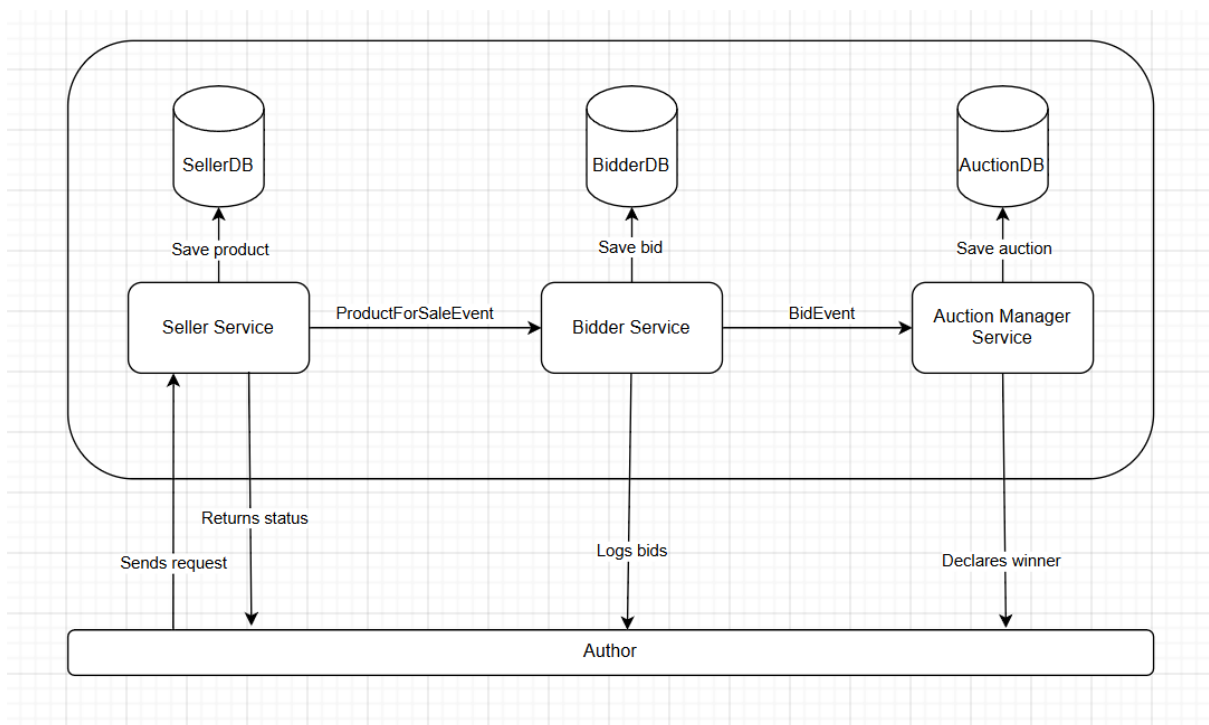


Figure 2 - System architecture

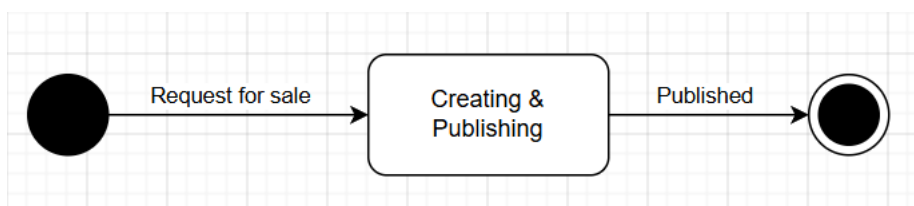


Figure 3 - Seller Service State Machine

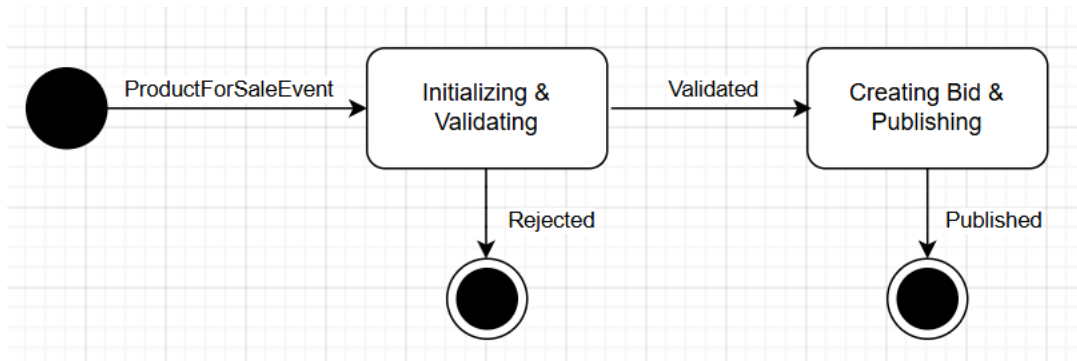


Figure 4 - Bidder Service State Machine

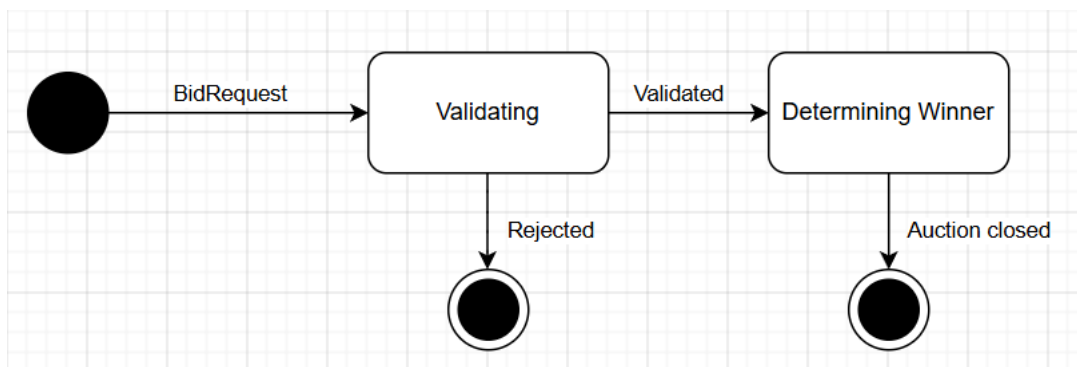


Figure 5 - Auction Manager Service State Machine

From monolith to Microservices

For migrating services:

Seller-service: The chosen pattern to migrate this service would be strangler-fig. The main difference between strangler-fig and branch-by-abstraction is that in strangler-fig the proxy is created outside the monolith, whereas in branch-by-abstraction the abstraction is created inside the monolith. Seller-service is the entry point in the application and is called directly by the user request, therefore strangler-fig works for this service. A proxy is created outside the monolith, calling the old seller functionality. After the seller functionality is implemented as a microservice, the proxy can call the microservice instead.

Other patterns from the course and the reasons why they are not the best choices:

Parallel run: this pattern runs both the old and new implementations and ensures they provide the same results. Because the main responsibilities of seller-service are writing to database and sending messages to bidders, running both implementations simultaneously is not possible. Executing any of these functionalities twice would cause trouble. Mocking these functionalities would mean mocking the entire service functionality, therefore, other testing methods used outside production are preferable. Canary release is however possible, making only a few of the users use only the new implementation, while most users still use

the old implementation until the new implementation gradually replaces the old implementation for all users.

Decorating collaborator: this pattern is a workaround solution when the codebase cannot be modified and new functionality needs to be added to the monolith. For this project, the codebase can be modified and there is no reason to keep the monolith.

Bidder-service: The chosen pattern to migrate this service would be branch-by-abstraction. Bidder-service is called by the seller-service, therefore, in a monolith application, the bidder functionality would lie deep into the monolith. In this case, an abstraction can be created inside the monolith. The seller would call the abstraction and the abstraction would call the old implementation until the bidder functionality is implemented as a service. Then, the abstraction would call the new bidder-service.

For parallel-run and decorating-collaborator, the reasoning is the same as for seller-service. They are not recommended for this service either. Canary release is however possible.

Auction-manager-service: The chosen pattern to migrate this service would be branch-by-abstraction. Auction-manager-service is called by the bidder-service, therefore, in a monolith application, the auction-manager functionality would lie deep into the monolith.

For parallel-run and decorating-collaborator, the reasoning is the same as for seller-service. They are not recommended for this service either. Canary release is however possible.

For migrating databases: The chosen pattern would be migrating from a monolith shared database to a database-per-service model. Each microservice would get its own schema which ensures the highest degree of decoupling. The main advantage is that each microservice and its database can be deployed independently of others. If a service were to need data from a different service database, it would get the data from that service, not the database itself. This complete separation of schemas works for this project because the microservices work mostly independent from each other: the seller service contains its products, and does not know anything about the bids or auctions; the bidders do not need to alter products (from seller-service) and know nothing about auction aggregates; the auction-manager does not need to alter neither products, nor bids, saving only its auction results. Therefore, each service can send only the necessary information to the next service in the sequence.

Other patterns from the course and the reasons why they are not the best choices:

Split-tables: this pattern would work well as a substitute if the schemas could not be completely separated. For this project, the schemas can be separated.

Database-view: this pattern involves unnecessary copying of data and more coupling than separate schemas or split-tables.

Database-as-a-service: this pattern provides an unnecessary abstraction layer. For this project, each microservice can handle its own database directly.

Change-data-capture: this pattern is a workaround solution for projects where the codebase cannot be modified. For this project, the codebase can be modified.

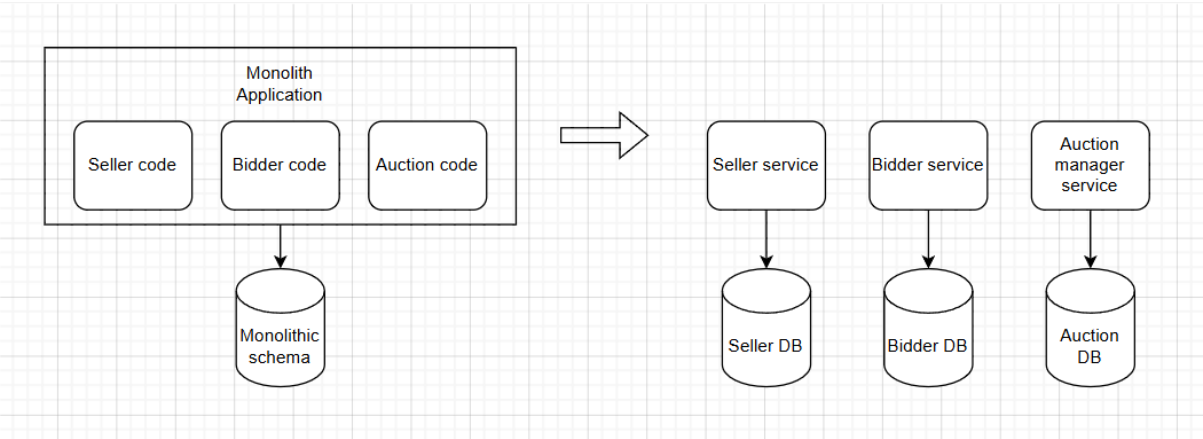


Figure 6 - Migration process

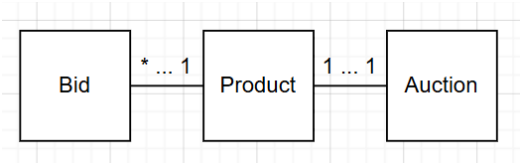


Figure 7 - Type of relationships

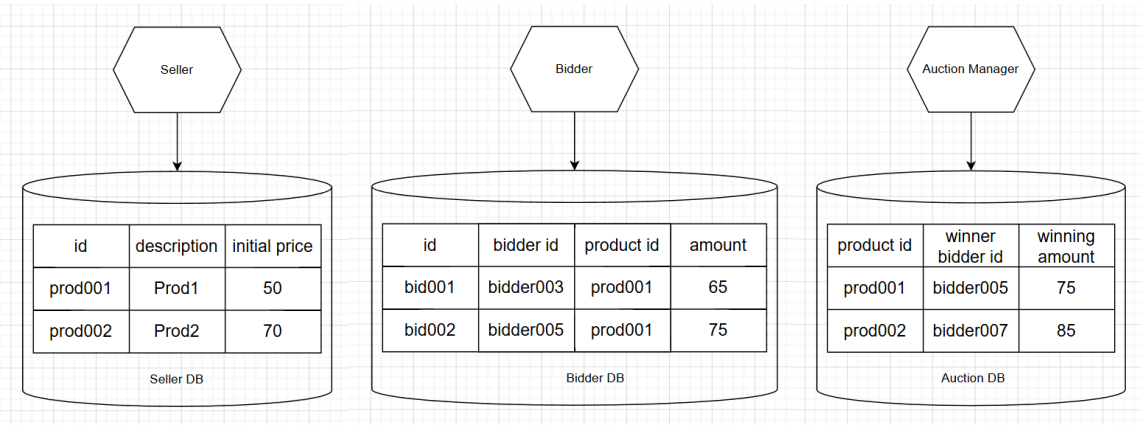


Figure 3 – Database schemas

Data communication structures

Selected workflow pattern: choreography

Bidding is intrinsically asynchronous, making Choreography the right choice for this project. This way, bidders make decisions in parallel, locally, independently of each other, each in its own time. The system becomes less coupled and more scalable as bidders can be added or removed. The system as a whole does not depend on a fixed number of bidders. This also increases fault tolerance, as offline bidders do not crash the system.

Orchestration would imply the existence of a bid-manager to coordinate all bidders and determine when they bid. This would increase coupling and create a single point of failure: if the orchestrator is down, the whole system is down. It also stands against the principles of bidding where bidders are independent.

Data communication pattern: event-driven-communication

Event-driven-communication is a natural choice for a bidding system based on choreography because events act as triggers. When the seller sends an event, the bidders react and when the bidders send an event, the auction-manager reacts. When the messages determine a change of state in another service, event-driven communication is more appropriate than publish-subscribe.

The seller needs to notify all bidders, regardless of how many they are. The bidders answer asynchronously. Event-driven-communication is more resilient in this case than other communication patterns such as request-response.

Request-response communication is not suited for this system because it is impractical to send a large number of requests and wait for their responses, when dealing with multiple independent services.

Event schemas for communication between microservices: Json

Event-driven-communication needs a communication schema that can be serialized and deserialized. Among the existing schemas, Json has a few advantages. It is a widely used and compatible format, natively supported by most existing frameworks, common in event-driven-communication and it is human-readable. It is tolerant and flexible: fields can be easily added or removed.

Product-For-Sale-Event fields: String productId, String description, int initialPrice.

Bid-Event fields: String bidderId, String productId, String description, int amount.